Sorting Algorithms

Java Enums

An enum is a special "class" that represents a group of **constants** (unchangeable variables, like final variables).

To create an enum, use the enum keyword (instead of class or interface), and separate the constants with a comma. Note that they should be in uppercase letters:

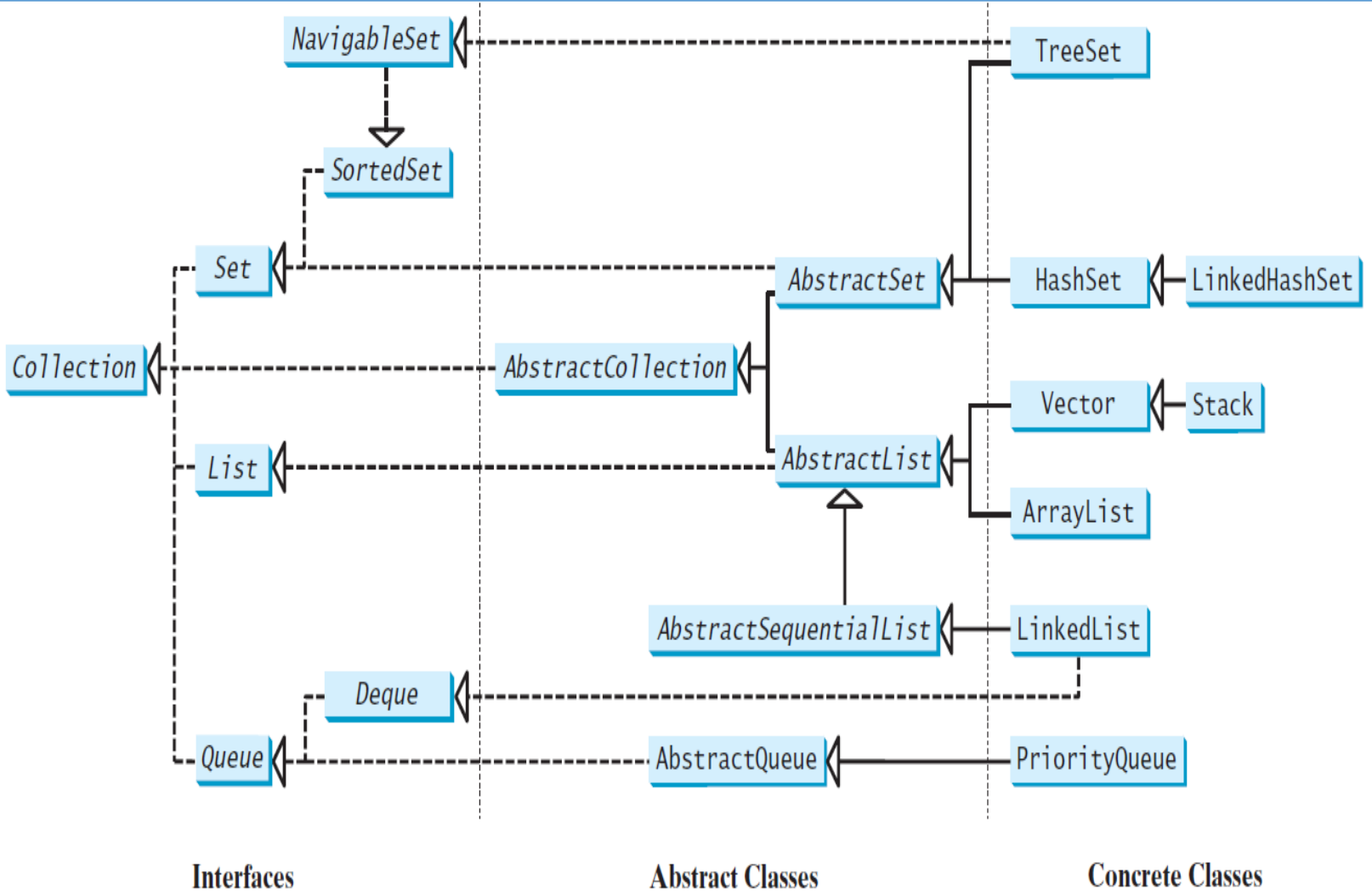
```
public class Main {
  enum Level {
    LOW,
    MEDIUM,
    HIGH
  }

  public static void main(String[] args) {
    Level myVar = Level.MEDIUM;
    System.out.println(myVar);
  }
}
```

Java Collections Framework

- A *collection* is a container object that holds a group of objects, often referred to as *elements*
- Collections has:
  - List
    - An ordered sequence accessed by position called an *index*
  - Set
    - An unordered collection of unique elements accessed by *key*
  - Map
    - A collection of key-value pairs accessed through the *key* to retrieve data stored as the *value*

Java Collections Framework



Collections Interface

- Root interface for manipulating a collection of objects

Methods:

- add
- addAll
- clear
- contains
- containsAll
- equals
- hashCode
- isEmpty
- remove
- removeAll
- retainAll
- size
- toArray

Methods inherited from **Iterator Interface:**

- hasNext
- next
- remove* (optional, implement at your own risk)

Iterator

- `public interface Iterator<E>`
- This interface is a member of the Java Collections Framework.
- By implementing this interface, you will be able to traverse through your data structure (e.g linked list) easily using a for-each loop.

- In order to write an Iterator for our linked list, there are two methods that we **must** override from the interface: `hasNext()` and `next()`.

`boolean hasNext()` – returns true if the iteration has more elements

`E next()` – returns the next element in the iteration (E is the type parameter)

List

Characteristics

- Interface
- An ordered collection known as a sequence
- Zero-based indexing where user can access the elements by index
- Member of Collections framework
- Allows user to specify where the element is stored

Some useful methods

- add(element)
- add(index,element)
- clear()
- contains()
- get()
- indexOf()
- isEmpty()
- remove()
- set()
- size()
- sublist()
- toArray()
- sort() with Comparator

Implemented by

- Set
- ArrayList
- LinkedList
- Vector
- AbstractList
- AbstractSequentialList

Array List

Hierarchy

- Object➔AbstractCollection➔AbstractList➔ArrayList

Characteristics

- resizeable
- null allowed
- Best use case: random access through an index without inserting or removing elements other than at the end

Methods

- trimToSize()

Runtimes

O(1)	size, isEmpty, get, set, iterator, listIterator
“amortized O(n)”	add—adding n elements requires O(n)
O(n)	all of the rest (roughly)

Linked List

Hierarchy

- Object➔AbstractCollection➔AbstractList➔ArraySequentialList

Characteristics

- doubly-linked list implementation of List and Deque (pronounced “deck”) interfaces
- null allowed
- traverse from beginning or end, whichever is closer
- can grow or shrink dynamically
- best use case: insertion or deletion from any place in the list

Methods

- addFirst()
- addLast()
- get()
- getFirst()
- getLast()
- pop()
- push()
- removeFirst()
- removeLast()

Sorting Algorithm

Algorithm	*Worst*	*In place? (Yes/No)*	*Stable? (Yes/No)*
Bubble			
Selection			
Insertion			
Shell*			
Merge			
Quick			
Heap			

Review - Swapping Algorithm

```
procedure swap(a, b)

   set temp to 0
   temp ← a
   a ← b      // a holds value of b
   b ← temp   // b holds value of a stored in temp

end procedure
```

Review - Complexity

- ***Complexity***
  - A measure of the computing resources that are used by a piece of code, such as time, memory, or disk space.
- ***Complexity classes***
  - Most frequently executed line of code
  - Determining the number of times it is executed
  - Extracting the highest power of N without coefficients

Stable Algorithm

Stable means that the order of input elements is ***unchanged*** except where change is required to satisfy the requirements.
A **stable sort** applied to a sequence of ***equal elements will not change their order***.

In-place sorting Algorithm

In-place

In-place means that no new storage space is allocated in-order for the algorithm to work.

```
public static void revereseArray(int []arr,
                                 int n)
{
    // Create a copy array
    // and store reversed
    // elements
    int []rev = new int[n];
    for (int i = 0; i < n; i++)
        rev[n - i - 1] = arr[i];

    // Now copy reversed
    // elements back to arr[]
    for (int i = 0; i < n; i++)
        arr[i] = rev[i];
}
```

```
static void rvereseArray(int arr[],
                         int start, int end)
{
    int temp;

    while (start < end)
    {
        temp = arr[start];
        arr[start] = arr[end];
        arr[end] = temp;
        start++;
        end--;
    }
}
```

1. Selection Sort

The selection sort algorithm sorts an array by repeatedly finding the minimum element (considering ascending order) from unsorted part and putting it at the beginning. The algorithm maintains two subarrays in a given array.

1) The subarray which is already sorted.

2) Remaining subarray which is unsorted.

In every iteration of selection sort, the minimum element (considering ascending order) from the unsorted subarray is picked and moved to the sorted subarray.

arr[] = 64 25 12 22 11

// Find the minimum element in arr[0...4]
// and place it at beginning

11 25 12 22 64

// Find the minimum element in arr[1...4]
// and place it at beginning of arr[1...4]
11 **12** 25 22 64

// Find the minimum element in arr[2...4]
// and place it at beginning of arr[2...4]
11 12 **22** 25 64

// Find the minimum element in arr[3...4]
// and place it at beginning of arr[3...4]
11 12 22 **25** 64

1. Selection Sort

- Best-case: $O(N^2)$
- Average-case: $O(N^2)$
- Worst-case: $O(N^2)$
- Stable: No *e.g. [2,2,1]*
- In-place: yes

```
for(i in range (0 to list size)) {
    smallestIndex = i;

    //unsorted part of list
    for(j in range (i+1 to list
size)){
        if (list[j] < smallest){
            smallestIndex = j
        }
    }

    swap(list[smallestIndex],list[i])
}
```

2. Bubble Sort

Bubble Sort is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements if they are in wrong order.

i = 0	j	0	1	2	3	4	5	6	7
	0	5	3	1	9	8	2	4	7
	1	3	5	1	9	8	2	4	7
	2	3	1	5	9	8	2	4	7
	3	3	1	5	9	8	2	4	7
	4	3	1	5	8	9	2	4	7
	5	3	1	5	8	2	9	4	7
	6	3	1	5	8	2	4	9	7
i=1	0	3	1	5	8	2	4	7	9
	1	1	3	5	8	2	4	7	
	2	1	3	5	8	2	4	7	
	3	1	3	5	8	2	4	7	
	4	1	3	5	2	8	4	7	
	5	1	3	5	2	4	8	7	
i=2	0	1	3	5	2	4	7	8	
	1	1	3	5	2	4	7		
	2	1	3	5	2	4	7		
	3	1	3	2	5	4	7		
	4	1	3	2	4	5	7		
i=3	0	1	3	2	4	5	7		
	1	1	3	2	4	5			
	2	1	2	3	4	5			
	3	1	2	3	4	5			
i=4	0	1	2	3	4	5			
	1	1	2	3	4				
	2	1	2	3	4				
i=5	0	1	2	3	4				
	1	1	2	3					
i=6	0	1	2	3					
		1	2						

First Pass:

(**5 1** 4 2 8) –> (**1 5** 4 2 8), Here, algorithm compares the first two elements, and swaps since 5 > 1.

(1 **5 4** 2 8) –> (1 **4 5** 2 8), Swap since 5 > 4

(1 4 **5 2** 8) –> (1 4 **2 5** 8), Swap since 5 > 2

(1 4 2 **5 8**) –> (1 4 2 **5 8**), Now, since these elements are already in order (8 > 5), algorithm does not swap them.

Second Pass:

(**1 4** 2 5 8) –> (**1 4** 2 5 8)

(1 **4 2** 5 8) –> (1 **2 4** 5 8), Swap since 4 > 2

(1 2 **4 5** 8) –> (1 2 **4 5** 8)

(1 2 4 **5 8**) –> (1 2 4 **5 8**)

Now, the array is already sorted, but our algorithm does not know if it is completed. The algorithm needs one **whole** pass without **any** swap to know it is sorted.

Third Pass:

(**1 2** 4 5 8) –> (**1 2** 4 5 8)

(1 **2 4** 5 8) –> (1 **2 4** 5 8)

(1 2 **4 5** 8) –> (1 2 **4 5** 8)

(1 2 4 **5 8**) –> (1 2 4 **5 8**)

2. Bubble Sort

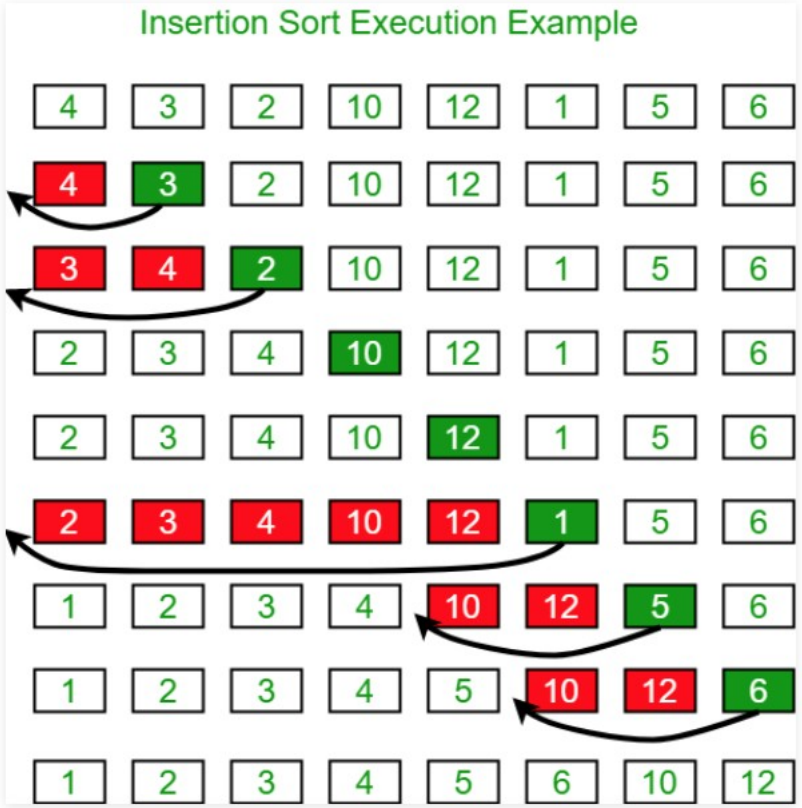
- Best-case: $O(N)$
- Average-case: $O(N^2)$
- Worst-case: $O(N^2)$
- Stable: yes
- In-place: yes

```
BUBBLESORT(A)
1 for i = 1 to A.length - 1
2     for j = A.length downto i + 1
3         if A[j] < A[j - 1]
4             exchange A[j] with A[j - 1]
```

```
boolean swapped;
for (i in range (0,list.length-1)) {
    swapped = false;
    for (j in range(list.length -i-1))  {
        if (arr[j] > arr[j + 1]) {
            swap(arr[j],arr[j+1])
            swapped = true;
        }
    }
    // if no two elements were
    // swapped by inner loop, then break
    if (swapped == false)
        break;
}
```

3. Insertion Sort

Insertion sort is a simple sorting algorithm that works similar to the way you sort playing cards in your hands. The array is virtually split into a sorted and an unsorted part. Values from the unsorted part are picked and placed at the correct position in the sorted part.



Algorithm

To sort an array of size n in ascending order:
1: Iterate from arr[1] to arr[n] over the array.
2: Compare the current element (key) to its predecessor.
3: If the key element is smaller than its predecessor, compare it to the elements before. Move the greater elements one position up to make space for the swapped element.

3. Insertion Sort

- Best-case: $O(N)$
- Average-case: $O(N^2)$
- Worst-case: $O(N^2)$
- Stable: yes
- In-place: yes

```
for(i in range (1 to list size)) {
    j = i;

    //Insert list[i] into sorted part
    //stop when list[i] is greater
    //than its left value
    while(j > 0 and list[j] < list[j-
1]) {
        swap(list[j],list[j-1])
        j--
    }
}
```